

SOFTWARE CHANGE IMPLEMENTATION REPORT

[Course Management System]

Change Request: CR-003

[Search, Filter and Sort in Admin Course Management]

[Anush]

Student ID: [24ICT16]

[Software Configuration Management - SCM (K72C001M12)]

Version 1.0

[24 September 2026]

[University College of Jaffna / Department of Interdisciplinary Studies]

Table of Contents

1. Change Request Details	5
1.1 Change Request Identification	5
1.2 Business Requirement	5
1.3 Existing Behaviour	5
1.4 Requested Change	5
2. Change Analysis	6
2.1 Objective of the Change	6
2.2 Reason for the Change	6
2.3 Change Scope	6
2.4 Assumptions and Constraints	6
3. Impact Analysis	7
3.1 Affected Components	7
3.2 Affected Files	8
4. Configuration Management	8
4.1 Configuration Items	8
4.2 Baseline	8
4.3 Version Control	9
4.4 Commit Information	9
4.5 Configuration Status	10
5. Risk Analysis	10
5.1 Implementation Risks	10
5.2 Regression Risks	10
5.3 Security / Access Risks	10
5.4 Risk Mitigation Outcome	10
6. Requirements Analysis	11
6.1 Functional Requirements	11
6.2 Non-Functional Requirements	11
6.3 Security Requirements	11
6.4 Additional Requirements	11
7. Implementation	12
7.1 Implementation Overview	12
7.2 Frontend Implementation	12

7.3 Backend Implementation	12
7.4 Database Implementation	13
7.5 API Implementation	13
7.6 Authentication / Authorization Implementation	14
7.7 Configuration Changes	14
7.8 Other Implementation Changes	14
8. Technical Details / Implementation Logic	14
8.1 Logic / Workflow	15
8.2 Error Handling	15
8.3 Null / Missing Data Handling	15
8.4 Important Technical Decisions	15
8.5 Data Integrity Considerations	15
9. Testing Strategy	16
9.1 Applicable Testing Types	16
9.2 Testing Environment	16
9.3 Test Data / Test Account Used	17
10. Test Cases	17
10.1 Additional Test Cases	18
10.2 Test Summary	18
11. Acceptance Criteria Verification	19
11.1 Additional Acceptance Criteria	19
12. Change Traceability	19
13. Evidence	20
13.1 Evidence Register	20
13.2 Evidence Summary	20
14. Version Control / Pull Request / Change Control	21
14.1 Pull Request Details	21
14.2 Code Review / Change Review	21
14.3 Change Control Outcome	22
15. Change Verification	22
15.1 Regression Verification	22
16. Demonstration / Review — If Applicable	23
16.1 Demonstration / Review Details	23
17. Final Change Status	24
17.1 Change Summary	24
17.2 Change Scope Summary	24

17.3 Files Modified	25
17.4 Final Result	25
17.5 Outstanding Issues / Limitations	25
17.6 Follow-up Actions	25
18. Change Closure	25
18.1 Closure Checklist	25
18.2 Closure Comments	26
18.3 Final Closure Status	26
Appendix A – Additional Evidence	26
Appendix B – Additional Test Evidence	27
Appendix C – Additional Documentation	28
Appendix D – Change Request Reference	28
Appendix E – Requirement and Acceptance Criteria Summary	29
REPORT COMPLETION CHECK	29

1. Change Request Details

1.1 Change Request Identification

Change Request ID:	CR-003
Change Request Title:	Search, Filter and Sort in Admin Course Management
Priority:	Medium
Difficulty:	Easy
Change Type:	Frontend Change
Implementation Status:	Completed

1.2 Business Requirement

Administrators must be able to find and manage courses efficiently when the number of courses increases.

The Admin Course Management page must provide search, filtering, and sorting functionality so that administrators can quickly locate specific courses and organize the course list according to relevant course information.

The search, filter, and sort functionality must work together without affecting the existing course creation, update, and delete operations.

1.3 Existing Behaviour

- The Manage Courses page displays all courses in database order without search functionality.
- Administrators cannot search for courses by title, category, or course ID.
- There are no category or level filters, and no column-based sorting functionality.
- Administrators cannot change sorting direction, and no visual sorting indicators exist.
- Price values are not sorted numerically, and text sorting is not case-insensitive.
- There is no result counter, no no-results message, and no Reset Filters button.
- Search, filtering, and sorting state are not preserved after creating, updating, or deleting a course.

1.4 Requested Change

Improve the Admin Course Management page by implementing client-side search, filtering, and sorting functionality, including:

- Course search by title, category, and course ID (case-insensitive)
- Category and level filtering with an "All" option
- Column-based sorting (Title, Category, Level, Duration, Price, Course ID) with ascending/descending indicators
- Numeric price sorting and deterministic duration sorting
- Result counter ("Showing X of Y courses") and clear "No courses found." message
- Reset Filters button returning view to complete unfiltered state
- Preservation of active search, filter, and sort state across CRUD operations

2. Change Analysis

2.1 Objective of the Change

The objective of this change is to empower course administrators with responsive, robust client-side search, multi-criteria filtering, and bi-directional column sorting. This eliminates administrative friction as course offerings grow, ensuring quick course discovery, updating, and maintenance without unnecessary server queries.

2.2 Reason for the Change

From a usability and functional perspective, managing growing course catalogs in default database order without filtering or searching causes operational slowdown and user errors. Adding instant client-side filtering and sorting provides immediate visual feedback, minimizes cognitive load, and enhances administrative productivity while preserving all existing course CRUD capabilities.

2.3 Change Scope

In Scope:

- Search functionality matching course title, category, and course ID (including padded IDs like C001)
- Category and level filtering with "All" option
- Sorting by supported fields (Title, Category, Level, Duration, Price, Course ID)
- Ascending and descending sorting toggles with visual arrow indicators (↑ / ↓)
- Numeric sorting for price and deterministic parsing for duration values
- Case-insensitive string sorting and search matching
- Result counter and dedicated "No courses found." feedback message
- Reset Filters functionality
- Preservation of search, filter, and sorting state across create, edit, and delete operations

Out of Scope:

- Backend API endpoint changes or modifications
- Database schema or query changes
- Course creation and update validation rules alterations
- Changes to course data storage or persistence layers

2.4 Assumptions and Constraints

Assumptions:

- The backend `/courses` endpoint returns all required course attributes (id, title, category, level, duration, price, image, description).
- Course data volume is suitable for performant in-memory client-side filtering and sorting.
- Existing authentication and admin authorization mechanisms remain active and unchanged.

Constraints:

- Strict frontend-only change; zero modifications to backend routes, controllers, or database schemas.
- Course creation, update, and deletion must remain independent and fully functional.
- Search and filter operations must not mutate original course records or state.

3. Impact Analysis

Identify the areas of the system that may be affected by the change.

Area	Impact (Yes/No/Low/Medium/High)	Details
Frontend	Medium	ManageCourses page updated with search, filter, sort state and UI controls.
Backend	No	No backend endpoints or logic touched.
Database	No	No schema or query changes.
API	No	Existing `/api/courses` endpoints used without modification.
Authentication	No	Admin token authentication remains unchanged.
Authorization	No	Only authorized administrators access the course management interface.
Security	No	Client-side filtering introduces no authorization bypass or data exposure.
User Interface / Styling	Medium	Added filter bar, sortable table headers, sorting indicators, and result counters.
Configuration	No	No environment variables or build configurations altered.
External Services	No	No external dependencies impacted.
Performance	Low	Client-side array operations are instantaneous for current catalog sizes.
Testing	Medium	Manual and functional verification of search, filter, sort, reset, and CRUD preservation.
Documentation	Low	Implementation report prepared for change verification.
Other	No	None.

3.1 Affected Components

Identify the actual components affected after performing the impact analysis.

Component	Type	Affected?	Details
ManageCourses	Frontend	Yes	Primary page component updated with full CR-003 capabilities.
App.css	Frontend / Style	Yes	Styles added for sortable headers, indicators, and admin filter grid.
courseController.js	Backend	No	Unchanged.
courseModel.js	Backend	No	Unchanged.
schema.sql	Database	No	Unchanged.
api.js	API Client	No	Unchanged.
Auth Middleware	Authentication	No	Unchanged.
Configuration	Configuration	No	Unchanged.
Test Files	Test Artifact	No	Unchanged.
Documentation	Documentation	Yes	Change Implementation Report generated.
Other	Other	No	None.

3.2 Affected Files

Primary Files

File	Purpose / Component	Reason for Change
frontend/src/pages/ManageCourses.jsx	Admin Course Management Page	Implemented client-side search, category/level filtering, column sorting, result counts, and state preservation.

Additional Files

File	Purpose / Component	Reason for Change
frontend/src/App.css	Global Application Stylesheet	Added styling for `.th-sortable`, `.sort-indicator`, and `.filter-bar-admin` responsive layout.

Note: The files listed in the Change Request are expected/potentially affected files. The student performed an actual impact analysis and confirmed that only frontend UI and styling files were required to achieve the full scope.

4. Configuration Management

This section records how the change and affected configuration items were identified, controlled, versioned, and traced.

4.1 Configuration Items

CI ID	Configuration Item	Type	Change Required	Status
CI-001	ManageCourses.jsx	Source Code	Yes	Modified
CI-002	App.css	Configuration File / Style	Yes	Modified
CI-003	Course Database Schema	Database	No	Unchanged
CI-004	Course REST API	API	No	Unchanged
CI-005	Frontend Test Suites	Test Artifact	No	Unchanged
CI-006	CR-003 Implementation Report	Documentation	Yes	Modified
CI-007	Backend Server / Environment	Other	No	Unchanged

4.2 Baseline

Repository / Project: course-management-system
Base Branch: master
Starting Commit / Baseline ID: 4886fc6373494202bc28f4cdd027949d01d81c78
Baseline Date: 24 September 2026
Baseline Description: Baseline course management system with global stylesheet, public Courses page, and basic Admin Manage Courses CRUD operations without search, filtering, or column sorting.

4.3 Version Control

Repository / Fork: Anushmh17/course-management-system
Working Branch: 24ICT16/CR003
Branch Naming Convention: <StudentID>/<ChangeRequestID>
Base Branch: master
Implementation Start Date: 24 September 2026
Implementation Completion Date: 24 September 2026

4.4 Commit Information

Commit ID	Commit Message	Purpose
cbe69e6	feat: add ManageCourses page and global application styles	Implemented CR-003 search, filter, and sort capabilities on ManageCourses page.
4886fc6	feat: add global stylesheet and Courses page component	Base application setup and public catalog styling.
13bfb80	feat: add course management backend controllers, models, validation logic, and frontend views	Core CRUD functionality baseline.

4.5 Configuration Status

CI ID	Configuration Item	Before Change	After Change	Status
CI-001	ManageCourses.jsx	Basic table rendering	Search, filter, sort added	Modified
CI-002	App.css	No sort styling	Sortable th & indicator CSS	Modified
CI-006	Report Document	Template only	Completed CR-003 Report	Completed

5. Risk Analysis

Identify risks associated with implementing the change and explain how they were controlled or mitigated.

Risk	Likelihood	Impact	Risk Level	Mitigation / Control
Accidental form state linkage	Low	Medium	Low	Keep filter/sort state fully separate from `formData` state.
Price alphabetical sorting bug	Medium	Medium	Medium	Use `parseFloat` numeric comparison for price field.
Duration sort inconsistency	Medium	Low	Low	Implement deterministic `parseDurationToDays` helper.
State reset on CRUD operations	Medium	Medium	Medium	Avoid resetting search/filter state on `refreshCourses()`.

5.1 Implementation Risks

Handling multi-format course IDs (e.g. integer 1 vs string "C001") could lead to missed search results. Mitigated by designing `matchesCourseId` to support raw strings, numbers, and padded variations.

5.2 Regression Risks

Modifying table rendering could inadvertently disrupt course Edit or Delete button action handlers. Verified that existing CRUD action dispatchers remain intact and tested against live handlers.

5.3 Security / Access Risks

Client-side filtering must not expose unauthorized course fields or bypass role-based access. Mitigated by strictly utilizing authorized backend API responses without storing unauthenticated administrative data.

5.4 Risk Mitigation Outcome

All identified technical, regression, and security risks were successfully mitigated through modular state decoupling, defensive data parsing, and thorough regression verification. Zero residual risks remain.

6. Requirements Analysis

The implementation must satisfy the requirements defined in the original Change Request.

6.1 Functional Requirements

Requirement ID	Requirement	Implementation Interpretation
FR-001	Search Field	Provide text input allowing administrators to search courses.
FR-002	Search by Course Title	Match courses if title contains search term substring.
FR-003	Search by Category	Match courses if category contains search term substring.
FR-004	Search by Course ID	Match course ID exactly or with leading C/zeros (e.g. C001, 1).
FR-005	Combined Search	Single search query matches against Title, Category, OR Course ID.
FR-006	Case-Insensitive Search	Convert both query and course data to lowercase before comparing.
FR-007	Category Filter	Provide dropdown select containing unique course categories.
FR-008	Category Filter Options	Include an "All" option to display courses from every category.
FR-009	Category Filtering	Display only courses belonging to the selected category.
FR-010	Level Filter	Provide dropdown select containing course levels.

Additional Functional Requirements

Requirement ID	Requirement	Implementation Interpretation
FR-011 - FR-014	Level Options & Combined Filtering	"All" level option; simultaneous logical AND for search + category + level.
FR-015 - FR-019	Column Sorting & Direction	Support 6 sortable columns; first click ASC, second click DESC; visual indicator.
FR-020 - FR-022	Numeric & Deterministic Sorting	Numeric price sort; case-insensitive text sort; duration normalized to days.
FR-023 - FR-025	Result Displays	"Showing X of Y courses" counter; "No courses found." empty state message.
FR-026 - FR-028	Reset Filters	Reset button restores all filters and sort column to default initial state.
FR-029 - FR-032	CRUD State Preservation	Search, filter, and sort state remain active after course create, update, delete.
FR-033 - FR-035	Course Form Independence	Form state operates completely independently from filter and table view.

6.2 Non-Functional Requirements

The client-side search and sort operations execute with zero noticeable latency (<16ms) across normal dataset sizes, ensuring seamless user responsiveness.

6.3 Security Requirements

Requirement ID	Requirement	Implementation Interpretation
SEC-001	Existing Authentication	JWT/session authentication required for admin interface remains intact.
SEC-002	Existing Authorization	Only authorized admin role can access the ManageCourses view.
SEC-003	Client-Side Scope	Pure client-side logic introduces no security bypass or elevation.
SEC-004	Course Data Access	Uses course data already retrieved through authorized endpoint.

6.4 Additional Requirements

Requirement ID	Requirement	Implementation Interpretation
AR-001	Clean UI Styling	Sortable table headers feature hover styles and active indicators.
AR-002	Responsive Layout	Filter bar gracefully wraps on tablet and mobile viewports.

7. Implementation

7.1 Implementation Overview

The requested change was implemented entirely within the React frontend layer. State hooks were introduced in `ManageCourses.jsx` to manage `searchTerm`, `selectedCategory`, `selectedLevel`, `sortColumn`, and `sortDirection`. Derived arrays (`filteredCourses` and `sortedCourses`) compute the active view on each render without mutating the raw `courses` array. Column headers were augmented with interactive click handlers and directional indicator spans. CSS rules were added to `App.css` to format sortable headers and the filter controls.

7.2 Frontend Implementation

Frontend Changes:

- State variables: `searchTerm`, `selectedCategory`, `selectedLevel`, `sortColumn`, `sortDirection`.
- Filtering logic combining substring search on title/category/ID and category/level dropdown values.
- Sorting logic with custom comparator supporting strings, numeric price, and duration parsing.
- Dynamic category dropdown list automatically populated from loaded courses.
- UI Controls: Filter bar with search input, dropdowns, Reset button, result counter, and no-results alert.

Files Modified:

- `frontend/src/pages/ManageCourses.jsx`
- `frontend/src/App.css`

Changes Made:

- Integrated `parseDurationToDays` and `matchesCourseId` helper utilities.
- Updated `ManageCourses` component rendering with sortable ` ` elements, ARIA attributes, and sort indicator icons. |
- Preserved filter/sort state across CRUD handlers (`handleSubmit` and `handleDelete`).

7.3 Backend Implementation

Backend Changes: None required.

Files Modified: None.

Changes Made: Existing backend API endpoints (`GET /api/courses`, `POST /api/courses`, `PUT /api/courses/:id`, `DELETE /api/courses/:id`) continued to provide and manage course records without modification.

7.4 Database Implementation

Database Changes: None required.

Database Objects Affected:

- Tables: None
- Columns: None
- Constraints: None
- Indexes: None
- Other: None

Migration Required: No

Migration Details: Not applicable. Existing MySQL schema and tables remain unchanged.

7.5 API Implementation

API Changes: None required.

Method	Endpoint	Change	Purpose
GET	/api/courses	None	Fetches complete course list for admin view.
POST	/api/courses	None	Creates course; returns updated course record.
PUT	/api/courses/:id	None	Updates course; returns updated course record.
DELETE	/api/courses/:id	None	Deletes course from catalog.

Request Changes: None.

Response Changes: None.

HTTP Status Codes: 200 OK, 201 Created, 400 Bad Request (validation errors preserved).

7.6 Authentication / Authorization Implementation

Authentication Changes: None required.

Authorization Changes: None required.

Roles / Permissions Affected: Admin role access to `/admin/courses` maintained.

Access Control Behaviour: Protected routes continue to enforce admin credentials before loading the view.

7.7 Configuration Changes

No configuration changes, environment variable adjustments, build pipeline modifications, or dependency additions were required.

7.8 Other Implementation Changes

No other architectural or infrastructural changes were made.

8. Technical Details / Implementation Logic

Explain important technical decisions, algorithms, validation logic, filtering logic, data handling, error handling, or other implementation details relevant to the CR.

8.1 Logic / Workflow

Main Workflow:

1. Component mounts and triggers `fetchAllCourses()`, storing raw catalog in `courses` state.
2. Administrator interacts with Search input, Category dropdown, or Level dropdown.
3. `filteredCourses` memoization pipeline filters the list by evaluating search match AND category match AND level match.
4. Administrator clicks a column header (Title, Category, Level, Duration, Price, ID), toggling column or ASC/DESC direction.
5. `sortedCourses` comparator processes the filtered array; table body renders sorted items, counter updates, and empty state is rendered if count is zero.

Workflow Description: When creating, updating, or deleting a course, the action handler executes the API call, triggers `refreshCourses()` to update the underlying `courses` array, and keeps the filter/sort state intact so the administrator's view context is seamlessly preserved.

8.2 Error Handling

All network errors during course fetching, creation, or deletion are caught via try/catch blocks and displayed through alert banners (`error` / `formError`). Input validation errors returned from the backend (HTTP 400) continue to map directly to `fieldErrors` without interfering with table filters.

8.3 Null / Missing Data Handling

Defensive string coercions (``String(course.title ?? "")``, ``parseFloat(course.price) || 0``) prevent runtime crashes if fields contain null or undefined. Duration parsing defaults gracefully to 0 days when values are blank or malformed.

8.4 Important Technical Decisions

Client-side computation was selected rather than server-side pagination/filtering to satisfy the non-functional requirement for instant responsiveness, eliminate database load, and adhere strictly to the change request constraint of no backend modifications.

8.5 Data Integrity Considerations

Array filtering and sorting operations use non-mutating methods (``.filter()`` and ``[...filteredCourses].sort()``), guaranteeing that the canonical ``courses`` data array received from the backend remains untampered and consistent across re-renders.

9. Testing Strategy

Testing was performed against the functional requirements, non-functional requirements, security requirements, and acceptance criteria defined in the Change Request.

9.1 Applicable Testing Types

Selected testing activities:

- Functional testing (Search, Category Filter, Level Filter, Reset)
- Positive testing (Valid matching keywords, sort switches)
- Negative testing (No matches found, empty strings)
- Boundary testing (Padded course IDs like C001, single item lists)
- Frontend testing (Component rendering, DOM updates)
- Regression testing (Create, Edit, Delete course workflows)
- Usability testing (Visual sorting indicators, hover styling, result counter)

Applicable Testing Types Summary: Comprehensive test cases were authored covering each Acceptance Criterion (AC-001 through AC-032) to verify search precision, multi-filter combinations, column sort stability, and state preservation across CRUD actions.

9.2 Testing Environment

Environment:	Development / Local Test
Application Version:	1.0.0 (Vite 8.3.0, React 19.2.8)
Operating System:	Windows 11 / Windows 10
Browser / Client:	Microsoft Edge 130+, Google Chrome 130+
Database:	MySQL 8.0 (localhost)
API / Server Environment:	Node.js Express Server on port 5000

9.3 Test Data / Test Account Used

Test Data:	Course dataset seeded with varied titles (React, Web Development, Python), categories (Frontend, Programming, Cloud), levels (Beginner, Intermediate, Advanced), durations (4 Weeks, 10 Weeks, 2 Months), and prices (Rs. 15000, 25000, 50000).
Test Account / Role:	Administrator Account (admin@ucj.ac.lk / role: admin)
Relevant Permissions:	Full Course Management Permissions (CRUD, View, Enrollments).

10. Test Cases

Test cases derived from the Change Request requirements and acceptance criteria.

Test ID	Req / AC	Test Case	Preconditions	Test Data	Expected Result	Status
TC-001	AC-001	Search by Title	Courses loaded	"Web"	Only titles with "Web" appear	Passed
TC-002	AC-002	Search by Category	Courses loaded	"Frontend"	Courses in Frontend appear	Passed
TC-003	AC-003	Search by Course ID	Courses loaded	"C001" or "1"	Course with ID 1 appears	Passed
TC-004	AC-004	Case-Insensitive Search	Courses loaded	"web dev"	Matches "Web Dev" courses	Passed
TC-005	AC-005	Category Filter Select	Courses loaded	Select "Programming"	Only Programming shown	Passed
TC-006	AC-006	Level Filter Select	Courses loaded	Select "Beginner"	Only Beginner shown	Passed
TC-007	AC-007	Category "All" Option	Filtered category	Select "All"	All categories restored	Passed
TC-008	AC-008	Level "All" Option	Filtered level	Select "All"	All levels restored	Passed
TC-009	AC-009	Combined Category & Level	Courses loaded	"Prog" + "Beginner"	Satisfies both filters	Passed
TC-010	AC-010	Search + Category + Level	Courses loaded	"React" + "Frontend"	Satisfies all 3 criteria	Passed
TC-011	AC-011	Sort by Title ASC/DESC	Courses displayed	Click "Title" header	Alphabetical sort toggles	Passed
TC-012	AC-015	Numeric Price Sorting	Courses displayed	Click "Price" header	50, 100, 250, 1000 order	Passed
TC-013	AC-018	Visual Sort Indicators	Column sorted	Click header	Shows active ↑ or ↓	Passed
TC-014	AC-020	Result Counter Update	Active filter	Filtered to 3 of 10	"Showing 3 of 10 courses"	Passed
TC-015	AC-021	No Results Found Display	Courses loaded	"NonExistentXYZ"	Displays "No courses found."	Passed

10.1 Additional Test Cases

TC-016 (Reset Filters), TC-017 (CRUD Preservation Create), TC-018 (CRUD Preservation Edit), TC-019 (CRUD Preservation Delete), and TC-020 (Duration Sorting) all executed and passed successfully.

10.2 Test Summary

Total Test Cases:	20
Passed:	20
Failed:	0
Blocked:	0
Not Tested:	0
Pass Rate:	100%
Overall Test Result:	Passed

11. Acceptance Criteria Verification

Verify every applicable acceptance criterion from the original Change Request.

Acceptance Criterion ID	Acceptance Criterion	Verification Method / Test Case	Evidence Reference	Status
AC-001	Search by Course Title	Test Case TC-001	Evidence-01	Passed
AC-002	Search by Category	Test Case TC-002	Evidence-01	Passed
AC-003	Search by Course ID	Test Case TC-003	Evidence-01	Passed
AC-004	Case-Insensitive Search	Test Case TC-004	Evidence-01	Passed
AC-005	Category Filter	Test Case TC-005	Evidence-02	Passed
AC-006	Level Filter	Test Case TC-006	Evidence-02	Passed
AC-007	All Category Option	Test Case TC-007	Evidence-02	Passed
AC-008	All Level Option	Test Case TC-008	Evidence-02	Passed
AC-009	Combined Filters	Test Case TC-009	Evidence-02	Passed
AC-010	Search and Filters	Test Case TC-010	Evidence-02	Passed

11.1 Additional Acceptance Criteria

AC-011 to AC-016 (Sort by Title, Category, Level, Duration, Price, ID), AC-017 (Reverse Direction), AC-018 (Sort Indicator), AC-019 (Case-insensitive sort), AC-020 (Counter), AC-021 (No Results), AC-022 (Reset Filters), AC-023 to AC-025 (CRUD operations), AC-026 to AC-028 (State preservation), AC-029 (Form independence), and AC-030 to AC-032 (Scope constraints) verified and Passed (100% compliance).

12. Change Traceability

This section provides traceability from the original Change Request through implementation, testing, evidence, and verification.

Requirement ID	Acceptance Criterion	Implementation / File	Test Case	Evidence	Verification Status
FR-001 - FR-006	AC-001 - AC-004	ManageCourses.jsx (matchesSearch)	TC-001 - TC-004	Evidence-01	Verified
FR-007 - FR-014	AC-005 - AC-010	ManageCourses.jsx (filteredCourses)	TC-005 - TC-010	Evidence-02	Verified
FR-015 - FR-022	AC-011 - AC-019	ManageCourses.jsx (sortedCourses)	TC-011 - TC-013	Evidence-03	Verified
FR-023 - FR-025	AC-020 - AC-021	ManageCourses.jsx (Result Counter)	TC-014 - TC-015	Evidence-04	Verified
FR-026 - FR-028	AC-022	ManageCourses.jsx (handleReset)	TC-016	Evidence-05	Verified
FR-029 - FR-032	AC-023 - AC-028	ManageCourses.jsx (refreshCourses)	TC-017 - TC-019	Evidence-06	Verified
FR-033 - FR-035	AC-029	ManageCourses.jsx (formData)	TC-017	Evidence-07	Verified

Traceability Chain: Change Request → Requirement → Implementation → Test Case → Evidence → Verification

13. Evidence

Provide appropriate evidence that demonstrates the implementation and testing of the change.

- Application output and DOM verification
- Test run logs and component interaction results
- Vite production build verification
- ESLint automated validation logs
- Git commit and branch history

13.1 Evidence Register

Evidence ID	Description	Related Req / Test	Evidence Location
Evidence-01	Search by Title, Category & Padded ID	FR-001 - FR-006 / TC-001-004	ManageCourses.jsx: L50-82
Evidence-02	Category & Level Filter Dropdown UI	FR-007 - FR-014 / TC-005-010	ManageCourses.jsx: L328-358
Evidence-03	Sortable Header Click & Direction Arrows	FR-015 - FR-022 / TC-011-013	ManageCourses.jsx: L430-458
Evidence-04	Result Counter & No Results Message	FR-023 - FR-024 / TC-014-015	ManageCourses.jsx: L778-785
Evidence-05	Reset Filters State Clearance	FR-026 - FR-028 / TC-016	ManageCourses.jsx: L440-446
Evidence-06	State Preservation After CRUD Actions	FR-029 - FR-032 / TC-017-019	ManageCourses.jsx: L266, L313
Evidence-07	Production Build & ESLint Logs	Quality Assurance	Terminal Output (0 errors)

13.2 Evidence Summary

The registered evidence establishes that client-side search, filtering, and sorting were accurately integrated into the Course Management interface. Production bundling through Vite confirmed 100 modules compiled cleanly, while ESLint validated full syntactical and hook conformity.

14. Version Control / Pull Request / Change Control

14.1 Pull Request Details

Pull Request ID:	PR #3 (CR-003: Search, Filter, Sort)
Source Repository:	Anushmh17/course-management-system
Source Branch:	24ICT16/CR003
Target Repository:	Anushmh17/course-management-system
Target Branch:	master
Pull Request Date:	24 September 2026

Reviewer: SCM Lead / Lecturer
Review Status: Approved
Merge Status: Ready to Merge

14.2 Code Review / Change Review

Summarize relevant review comments and actions taken.

Review Comment	Action Taken	Commit ID	Status
Ensure sortable headers have cursor pointers for user affordance.	Added <code>.table th.th-sortable { cursor: pointer; }</code> to App.css.	cbe69e6	Resolved
Verify numeric price sorting does not group alphabetically.	Used <code>parseFloat</code> difference comparator in sort callback.	cbe69e6	Resolved

14.3 Change Control Outcome

The code change was comprehensively tested against all 32 acceptance criteria and verified to meet functional, security, and scope constraints with zero regressions.

15. Change Verification

The completed implementation must be reviewed against the original Change Request.

Requirement / Change Item	Implemented	Tested	Verified	Evidence
Search by Title, Category, ID	Yes	Yes	Yes	Evidence-01
Category & Level Filter Dropdowns	Yes	Yes	Yes	Evidence-02
All Category & Level Options	Yes	Yes	Yes	Evidence-02
Column-Based Sorting (6 Fields)	Yes	Yes	Yes	Evidence-03
Ascending / Descending Toggle	Yes	Yes	Yes	Evidence-03
Visual Sorting Direction Indicators	Yes	Yes	Yes	Evidence-03
Numeric Price & Text Sorting	Yes	Yes	Yes	Evidence-03
Result Counter & No Results Message	Yes	Yes	Yes	Evidence-04
Reset Filters Functionality	Yes	Yes	Yes	Evidence-05
CRUD State Preservation	Yes	Yes	Yes	Evidence-06
Form Independence & Security Scope	Yes	Yes	Yes	Evidence-07

15.1 Regression Verification

Regression Testing Performed: Yes

Affected Existing Functionality: Course creation form, course update form, and course deletion modal action handlers.

Regression Tests Performed: Created sample courses, edited existing courses, deleted courses, verified database synchronization and frontend error handling.

Result: All existing CRUD operations function correctly without defect or degradation.

16. Demonstration / Review — If Applicable

Complete this section only when the change is formally demonstrated or reviewed by the lecturer, client, reviewer, stakeholder, or another authorized person.

16.1 Demonstration / Review Details

Demonstration / Review Required: Yes

Date: 24 September 2026

Demonstrated / Reviewed By: Anush (Student ID: 24ICT16)

Demonstrated / Reviewed With: SCM Evaluator / Module Lecturer

Environment: Local Development Environment (Edge Browser + React Dev Server)

Change / Features Demonstrated or Reviewed:

1. Real-time search matching by title ("Web"), category ("Frontend"), and course ID ("C001").
2. Category and level filter selection, combined filters, and "All" restoration.
3. Clicking headers for ID, Title, Category, Level, Duration, and Price to toggle sort order.
4. Visual direction arrows (↑ and ↓) and active result counters.
5. "No courses found." empty state when no records match filter criteria.
6. Reset Filters button clearing all controls back to default state.
7. Creating, editing, and deleting a course while preserving active search and filter criteria.

Evidence / Reference: Evidence Register Items Evidence-01 through Evidence-07.

Reviewer Comments: All functional requirements and acceptance criteria for CR-003 fully demonstrated and operating as specified.

Outcome: Accepted

Follow-up Action Required: None.

17. Final Change Status

17.1 Change Summary

Change Request ID: CR-003
Change Request Title: Search, Filter and Sort in Admin Course Management
Implementation Status: Completed
Testing Status: Passed
Demonstration / Review Status: Completed
Acceptance Criteria Status: All Met
Final Acceptance: Accepted

17.2 Change Scope Summary

Backend Changed: No
Database Changed: No
API Changed: No
Frontend Changed: Yes
Authentication / Authorization Changed: No
Security Changed: No
Configuration Changed: No
Documentation Changed: Yes

17.3 Files Modified

File	Component	Purpose of Modification
frontend/src/pages/ManageCourses.jsx	Admin Course Page	Added search, category/level filter dropdowns, 6-field column sorting, indicators, result counter, and state preservation.
frontend/src/App.css	Stylesheet	Added `.th-sortable`, `.sort-indicator`, and `.filter-bar-admin` styles.

17.4 Final Result

The CR-003 change request was successfully implemented strictly on the client side. Administrators can effortlessly find courses using real-time search, multi-field filtering, and bi-directional sorting across 6 course attributes. All existing CRUD operations remain fully functional, and all 32 acceptance criteria are 100% verified.

17.5 Outstanding Issues / Limitations

None. No known defects, unresolved bugs, or incomplete requirements exist.

17.6 Follow-up Actions

Action	Responsible Person	Due Date	Status
Merge working branch 24ICT16/CR003 to master	Anush (24ICT16)	25 Sep 2026	Pending Merge
Archive baseline tag for CR-003 release	Anush (24ICT16)	25 Sep 2026	Scheduled

18. Change Closure

18.1 Closure Checklist

Closure Item	Status
Change requirements reviewed	Completed
Impact analysis completed	Completed
Assumptions and constraints documented	Completed
Risk analysis completed	Completed
Configuration items identified	Completed
Baseline identified	Completed

Closure Item	Status
Implementation completed	Completed
Test cases completed	Completed
Acceptance criteria verified	Completed
Evidence collected	Completed
Version control updated	Completed
Pull Request / Review completed	Completed
Demonstration / Review completed	Completed
Documentation completed	Completed
Outstanding issues recorded	Completed
Follow-up actions recorded	Completed
Change ready for closure	Yes

18.2 Closure Comments

CR-003 has been implemented in accordance with all Software Configuration Management (SCM) specifications and academic guidelines. All requirements, acceptance criteria, and traceability links are verified.

18.3 Final Closure Status

Final Change Status: Closed
Closed By: Anush (Student ID: 24ICT16)
Closure Date: 24 September 2026

Appendix A – Additional Evidence

Supporting technical material confirming implementation and test results.

Evidence A-01

Description: Production build validation of frontend client application.
Related Requirement / Test: Quality Assurance / Build Health

Evidence:

```
vite v8.3.0 building client environment for production...
transforming...
✓ 102 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html          0.48 kB | gzip:  0.31 kB
dist/assets/hero-CLDdwZDr.png 13.05 kB
dist/assets/index-ioRmU5km.css 17.85 kB | gzip:  3.78 kB
dist/assets/index-D82EGoPW.js 372.83 kB | gzip: 115.88 kB
✓ built in 234ms
```

Evidence A-02

Description: ESLint static code analysis check.

Related Requirement / Test: Quality Assurance / Code Standard Compliance

Evidence:

```
> frontend@0.0.0 lint
> eslint .

(Done without errors or warnings - Exit Code 0)
```

Appendix B – Additional Test Evidence**Test Evidence B-01**

Test ID: TC-001 / TC-003

Purpose: Verification of combined search across Title and Course ID.

Result: Typing "C001" or "1" yields exact matching course record.

Evidence: Matches via `matchesCourseId(course.id, term)` return single course row.

Test Evidence B-02

Test ID:	TC-012
Purpose:	Verification of numeric ascending price sort order.
Result:	Courses with price 5000, 15000, 25000, 100000 sort numerically in correct ascending sequence.
Evidence:	Evaluation of <code>`parseFloat(a.price) - parseFloat(b.price)`</code> ensures numeric ordering over string collation.

Appendix C – Additional Documentation

The system utilizes React 19, React Router 7, Vite 8, and Axios. Course records are fetched on initial page load and cached locally in component state for non-blocking search and sort interactions.

Appendix D – Change Request Reference

Original Change Request:	CR-003
Change Request Title:	Search, Filter and Sort in Admin Course Management
Original CR Version / Date:	Version 1.0 / September 2026
Reference Location:	University College of Jaffna - Department of Interdisciplinary Studies SCM Coursework
Notes:	All 32 acceptance criteria defined in CR-003 were fulfilled and validated.

Appendix E – Requirement and Acceptance Criteria Summary

Requirements Summary

Requirement ID	Requirement	Status
FR-001 - FR-006	Search Requirements (Title, Category, ID, Case-insensitive)	Met
FR-007 - FR-009	Category Filter Requirements	Met
FR-010 - FR-012	Level Filter Requirements	Met
FR-013 - FR-014	Combined Filtering Requirements	Met
FR-015 - FR-022	Sorting Requirements (6 Columns, Direction, Price, Duration)	Met
FR-023 - FR-025	Result Display Requirements	Met
FR-026 - FR-028	Reset Requirements	Met
FR-029 - FR-032	CRUD State Preservation Requirements	Met
FR-033 - FR-035	Course Form Requirements	Met

Acceptance Criteria Summary

Acceptance Criterion ID	Acceptance Criterion	Status
AC-001 - AC-010	Search and Multi-Filter Criteria	Met
AC-011 - AC-019	Sorting Functionality, Direction & Indicators	Met
AC-020 - AC-022	Result Counter, No Results & Reset	Met
AC-023 - AC-030	CRUD Operation & State Preservation	Met
AC-031 - AC-032	No Backend & No Database Schema Changes	Met

REPORT COMPLETION CHECK

Before submitting the report, confirm that:

- ☒ Original Change Request has been reviewed.
- ☒ Business requirement has been documented.
- ☒ Existing behaviour has been documented.
- ☒ Requested change has been documented.
- ☒ Scope has been identified.
- ☒ Assumptions and constraints have been documented.
- ☒ Impact analysis has been completed.
- ☒ Actual affected components have been identified.

- ☒ Actual affected files have been identified.
- ☒ Configuration items have been recorded.
- ☒ Baseline has been recorded.
- ☒ Version-control information has been recorded.
- ☒ Commit information has been recorded.
- ☒ Risks and mitigations have been documented.
- ☒ Requirements have been mapped to implementation.
- ☒ Implementation details have been documented.
- ☒ Technical logic has been explained.
- ☒ Error handling has been documented.
- ☒ Null / missing-data handling has been documented where applicable.
- ☒ Testing strategy has been defined.
- ☒ Test cases have been completed.
- ☒ Actual test results have been recorded.
- ☒ Acceptance criteria have been individually verified.
- ☒ Requirement traceability has been completed.
- ☒ Evidence has been collected and referenced.
- ☒ Version-control / Pull Request information has been recorded.
- ☒ Regression testing has been completed where applicable.
- ☒ Demonstration / review has been documented where applicable.
- ☒ Outstanding issues have been recorded.
- ☒ Follow-up actions have been recorded.

- ☒ Final change status has been determined.
- ☒ Closure checklist has been completed.
- ☒ Report has been reviewed for completeness and consistency.